Algorithmic Arbitrage on Polymarket: Infrastructure, Strategies, and Python Implementation

1. The Macroeconomics of Decentralized Prediction Markets

The proliferation of decentralized prediction markets has introduced a novel asset class characterized by binary options representing the probability of real-world events. Polymarket, operating predominantly on the Polygon blockchain (Chain ID: 137), has emerged as the definitive global venue for these conditional tokens.[1] By the fourth quarter of 2025, propelled by unprecedented global engagement during the United States electoral cycle, Polymarket reported monthly trading volumes exceeding three billion dollars, with sports markets subsequently capturing over sixty percent of total open interest.[3]

Unlike traditional decentralized finance automated market makers, Polymarket employs a hybrid-decentralized Central Limit Order Book architecture.[4] This complex microstructure facilitates off-chain order matching executed with high-throughput efficiency, coupled with non-custodial, atomic on-chain settlement via audited exchange smart contracts.[4] The operator of the matching engine possesses no capability to set prices or execute unauthorized trades, and users retain the fundamental cryptographic right to independently cancel orders directly on the blockchain.[4]

Because Polymarket shares are priced dynamically between zero dollars and one dollar, representing the aggregated market consensus regarding the percentage probability of a specific event occurring, pricing inefficiencies frequently arise across mutually exclusive outcomes.[6] The systematic pursuit and exploitation of these specific inefficiencies forms the basis of algorithmic prediction market arbitrage. The deployment of automated execution engines has fundamentally reshaped the platform, with sophisticated quantitative operators leveraging high-frequency scanning algorithms, machine-learning probability models, and Python-based execution infrastructures to extract consistent, market-neutral yields from transient mispricings.[7]

This comprehensive report details the theoretical, mathematical, and programmatic requirements necessary to engineer an institutional-grade Python trading bot capable of identifying and exploiting arbitrage opportunities on the Polymarket Central Limit Order Book. It exhaustively analyzes the underlying Conditional Token Framework, the mathematics of the dynamic fee implementations rolled out in early 2026, the specific smart contract infrastructure required for combinatorial multi-outcome events, and the definitive

implementation methodologies utilizing the official Python client.

2. Fundamental Topography of Decentralized Arbitrage

Arbitrage within the context of binary prediction markets differs fundamentally from classical spatial arbitrage observed across disparate cryptocurrency exchanges. Rather than relying on the purchase of a homogenous asset on a primary venue and its simultaneous sale on a secondary venue, prediction market arbitrage relies entirely on foundational probability theory and the immutable axiom that the sum of the probabilities of all mutually exclusive and exhaustive outcomes of an event must mathematically equal precisely one hundred percent, represented financially as exactly one dollar.[8]

2.1. Dutch Book Arbitrage Dynamics

The most direct and mathematically pure form of arbitrage on the Polymarket platform is known colloquially and academically as the Dutch Book. Every individual binary market on Polymarket invariably consists of a complementary "Yes" token and a "No" token representing the two potential states of the targeted condition.[10] At the exact moment of market resolution, driven by the Optimistic Oracle, the correct outcome token becomes strictly redeemable for exactly one USDC, while the incorrect token is simultaneously rendered fundamentally worthless.[11]

Consequently, the combined intrinsic value of exactly one "Yes" token and one "No" token for the exact same market condition is perpetually and structurally guaranteed to be exactly one dollar post-resolution. If the combined aggregated order book ask prices for both of these tokens fall below the one dollar threshold, a risk-free gross profit can be immediately locked in by purchasing both tokens simultaneously.[8]

An arbitrage opportunity exists if the lowest available ask price for the "Yes" token combined with the lowest available ask price for the "No" token, plus all associated execution costs and protocol fees, equates to a sum strictly less than one dollar. For instance, if the "Yes" token currently trades at an ask of forty-eight cents and the "No" token trades at an ask of forty-nine cents, the total capital outlay required to acquire both conditional tokens is exactly ninety-seven cents.[13] By holding both tokens in cryptographic custody until the final event resolution, the arbitrageur receives a guaranteed payout of one dollar, yielding a gross profit of three cents, which translates to a return on capital of slightly more than three percent prior to fee subtraction.[12]

Conversely, if the combined bids for the "Yes" and "No" tokens exceed one dollar, an arbitrageur who already holds inventory can sell both sides of the market simultaneously to lock in an immediate premium.[14] While theoretically infallible, the practical execution of Dutch Book arbitrage is a highly competitive endeavor. In highly liquid, mainstream markets, high-frequency

trading bots routinely scan and flatten these inefficiencies within milliseconds of their emergence.[12] As such, modern algorithmic strategies increasingly rely on identifying inefficiencies across much more complex multi-outcome events.

2.2. Combinatorial Arbitrage in Multi-Outcome Environments

A significant portion of the liquidity and open interest on Polymarket is concentrated within events that contain more than two distinct outcomes, such as electoral contests or multi-participant sporting tournaments. These are structured systematically as grouped market pairs nested under a single overarching event object.[10] In a mutually exclusive multi-outcome event comprising multiple potential candidates or results, the fundamental probabilistic rule remains identical: only one specific candidate can ultimately be declared the winner.

A combinatorial arbitrage opportunity arises when the mathematical sum of the "Yes" prices across all individual candidates in the event is calculated to be strictly less than one dollar, or when the implied probabilities of related sub-markets become temporarily decoupled due to localized liquidity shocks.[9] Researchers conducting empirical arbitrage analyses on historical Polymarket order book data have identified massive volumes of Combinatorial Arbitrage spanning multiple markets, with realized profit extraction estimates reaching forty million dollars.[9]

However, systematically executing combinatorial arbitrage by purchasing every single "Yes" outcome in an event featuring ten or more candidates is highly capital intensive and computationally complex. To resolve this inefficiency and facilitate highly complex multi-leg trading, Polymarket utilizes a sophisticated smart contract innovation known as the Negative Risk mechanism.[15]

2.3. Statistical Arbitrage and Machine Learning Applications

Beyond pure mathematical certainty, a growing cohort of technically sophisticated market participants generates substantial yields through statistical arbitrage methodologies. These traders deploy Python-based algorithms to scan hundreds of related markets simultaneously, searching for temporary divergences in historically correlated events.[7] For example, contracts tied to a specific presidential election outcome and contracts tied to legislative control outcomes may historically exhibit deep price correlation. When the price spreads between these markets widen abnormally, statistical arbitrageurs programmatically purchase the underpriced contract and sell the overpriced contract, closing the position to capture a yield once the prices inevitably converge back to their historical mean.[7]

The most advanced iterations of these execution bots utilize machine-learning probability models, where algorithms process massive flows of news data and social media sentiment to estimate the true fundamental odds of an event occurring.[7] When the model's derived probability heavily disagrees with the current Central Limit Order Book prices, the bot

systematically scales into positions, acting as a highly informed liquidity provider. Accounts utilizing these complex methodologies have publicly reported extracting profits ranging from ten thousand to nearly half a million dollars monthly.[7]

3. The Architecture of the Gnosis Conditional Token Framework

To architect a functional execution engine capable of navigating Polymarket's complex settlement infrastructure, a quantitative developer must deeply understand the on-chain token mechanics that power the entire ecosystem. All outcomes traded on the Polymarket platform are tokenized utilizing the Conditional Token Framework, an open-source smart contract standard originally developed by Gnosis.[11] Understanding these cryptographic operations is strictly mandatory for advanced trading strategies, institutional market making, and direct smart contract interactions via Python Web3 libraries.

3.1. Tokenization and Collateralization Mechanics

The Conditional Token Framework generates distinct ERC1155 tokens that mathematically represent the outcomes of prediction markets.[11] Every binary market fundamentally consists of two specific tokens: a "Yes" token that redeems for exactly one bridged USDC if the event occurs, and a "No" token that redeems for exactly one bridged USDC if the event definitively does not occur.[11] A foundational principle of the framework is that these tokens are always fully collateralized on-chain; every outstanding Yes and No pair is backed cryptographically by exactly one unit of USDC locked securely within the core smart contract.[11]

The framework provides three fundamental on-chain operations that underpin all arbitrage lifecycle events. The first operation is the Split, which allows an arbitrageur to convert units of USDC directly into corresponding pairs of Yes and No outcome tokens.[11] The second operation is the Merge, which allows a market maker holding an equal amount of Yes and No tokens to securely convert them back into base USDC collateral prior to market resolution.[11] The third and final operation is the Redeem function, which allows users holding the correct winning tokens to exchange them for USDC following the official resolution broadcast by the decentralized oracle.[11]

3.2. Cryptographic Token Identifiers

Each specific outcome token traded on the Central Limit Order Book possesses a unique position identifier, frequently referred to in the API documentation as the Token ID or Asset ID. This specific identifier is computed deterministically on the Polygon blockchain through a rigid three-step cryptographic process.[11]

The initial step involves computing the Condition ID. This is derived by hashing the address of the Universal Market Access oracle adapter, the specific question hash used for resolution

data, and the total outcome slot count, which is consistently set to two for all binary markets on the platform.[11]

The second step requires the computation of Collection IDs. The protocol derives this by hashing the parent collection identifier, the newly generated Condition ID, and a distinct index set.[11] The index set functions as a bitmask denoting exactly which outcome slots belong to a given collection. In a binary system, it represents either the first outcome or the second outcome.[11]

The third and final step generates the Position IDs. This is executed by hashing the address of the underlying collateral token, which is exclusively bridged USDC on the Polygon mainnet, alongside one of the corresponding collection identifiers derived in the previous computational step.[11] The two resulting cryptographic position identifiers represent the exact ERC1155 Token IDs utilized for trading the Yes and No outcomes on the off-chain matching engine.[11] While Python developers can compute these hashes manually using local cryptographic libraries for direct smart contract integration, the Token IDs are generally indexed and heavily cached by the Polymarket Gamma API, making them easily retrievable via simple REST GET requests.[11]

4. Negative Risk and Combinatorial Market Structures

As previously established, standard combinatorial arbitrage across multi-outcome events is highly inefficient regarding capital deployment. To resolve this structural limitation, Polymarket engineered the Negative Risk mechanism, a complex adaptation of the Conditional Token Framework that fundamentally alters the economic relationships between mutually exclusive tokens.[15]

4.1. The Mechanics of the Negative Risk Adapter

In a standard multi-outcome event lacking negative risk infrastructure, each individual market is entirely economically independent. If a trading bot identifies that a specific outcome is vastly overpriced and wishes to establish a short position, it is required to purchase that specific outcome's No tokens.[15] However, those No tokens possess absolutely no mathematical relationship to the other outcomes within the broader event.

The Negative Risk framework fundamentally modifies this paradigm. In an event explicitly flagged with the negative risk boolean, a No share in any specific market can be atomically converted directly into exactly one Yes share in every single other market contained within the overarching event hierarchy.[15] This complex multi-token conversion process occurs entirely on-chain through the invocation of the NegRiskAdapter smart contract, deployed on the Polygon mainnet at the address 0xd91E80cF2E7be2e162c6513ceD06f1dD0dA35296.[15]

The conversion mechanics are strictly atomic. If an arbitrage algorithm currently holds one No token representing a specific political candidate, the Python execution script can format and

transmit a transaction calling the specific convert function directly on the NegRiskAdapter contract interface.[15] Upon successful inclusion in a Polygon block, the No token is destroyed, and the algorithm's wallet immediately receives exactly one Yes token for every other participating candidate in the specified event.[15]

This mechanism is the cornerstone of capital-efficient combinatorial arbitrage. Betting definitively against one specific outcome is now economically and mathematically equivalent to betting simultaneously in favor of all other alternative outcomes.[15] If the aggregated Central Limit Order Book ask prices for the Yes tokens of Candidate A and Candidate B sum to ninety-five cents, but the No token for Candidate C is currently trading at an ask of ninety-two cents, the arbitrage algorithm can aggressively purchase the No token for Candidate C, execute the on-chain NegRiskAdapter conversion, and successfully secure the two required Yes tokens at a mathematically guaranteed discount of three cents.[15]

4.2. Augmented Negative Risk and Placeholder Ambiguity

An incredibly complex edge-case within the combinatorial arbitrage domain involves markets utilizing Augmented Negative Risk. Standard negative risk mechanisms require the complete, exhaustive set of all potential outcomes to be definitively known and statically hardcoded at the exact moment of market creation.[15] However, in highly dynamic, evolving events, such as early-stage political primaries or emerging technological races, new candidates or outcomes frequently materialize long after order book trading has commenced.

Augmented negative risk resolves this temporal issue by launching the event with a combination of explicitly named outcomes, reserved placeholder outcomes, and an explicitly defined "Other" outcome category designed to catch any result not specifically named.[15] When a new entity enters the event context, a pre-existing placeholder slot is officially clarified via the decentralized bulletin board.[15] As these placeholders are progressively assigned concrete identities, the semantic definition of the "Other" outcome inherently narrows.[15]

For Python arbitrage algorithms, navigating augmented negative risk markets is exceptionally dangerous without strict programmatic safeguards. Trading bots must be rigorously programmed to only execute orders on explicitly named outcomes.[15] Placeholder outcomes must be entirely ignored by the logic engine until they are definitively named, as the Polymarket user interface actively filters out unnamed outcomes and the matching engine liquidity is often highly erratic.[15] Furthermore, trading directly on the "Other" token in an augmented market carries severe, unquantifiable risk, as its underlying definition is actively mutating as placeholders are clarified.[15] An event is systematically identified as utilizing this dangerous structure when both the negRisk and enableNegRisk boolean flags return as true within the Gamma API JSON response payload.[15]

5. The 2026 Dynamic Fee Structure and Margin

Calculus

The long-term viability and net profitability of any Python-based prediction market arbitrage strategy are dictated entirely by execution costs. Throughout its initial growth phases and into the conclusion of 2025, Polymarket operated under a heavily subsidized zero-fee trading model, prioritizing user acquisition, deep liquidity, and global market share.[3] However, following the strategic acquisition of a licensed exchange and clearinghouse in late 2025, Polymarket initiated a fundamental shift in its execution economics.[3]

In a phased rollout beginning in January 2026, Polymarket systematically introduced a complex, dynamic taker fee model designed explicitly to fund a comprehensive Maker Rebate Program.[18] This fundamentally altered the mathematical equations governing Dutch Book arbitrage.

5.1. The Continuous Fee Formula

The taker fee implemented by the protocol is not a static flat percentage. Instead, it is calculated dynamically based on the exact implied probability, or price, of the shares being traded in the transaction. The fee architecture is designed to peak at the absolute highest point of statistical variance, which corresponds exactly to a fifty percent probability or a price of fifty cents.[18] As the price diverges from the midpoint and approaches the extreme bounds of zero or one dollar, the effective fee rate decays symmetrically toward zero percent.[18]

The fee is calculated programmatically using the following continuous mathematical function:

$$Fee = C \times p \times feeRate \times (p \times (1 - p))^{exponent}$$

Within this structural equation, C represents the total number of shares being traded, while p represents the exact execution price of the shares. The $feeRate$ serves as a market-specific baseline multiplier, and the $exponent$ acts as a curve modifier that shapes the steepness of the fee decay.[18]

The exact parameters utilized in the formula differ significantly based on the specific categorization of the market being traded.

Market Categorization	Implementation Date	Fee Rate Variable	Exponent Variable	Peak Effective Fee	Maker Rebate Allocation

Crypto Markets (5-Minute, 15-Minute)	Jan 19 & Feb 12, 2026	0.25	2	1.56%	20%
Crypto Markets (1H, 4H, Daily, Weekly)	March 6, 2026	0.25	2	1.56%	20%
Sports Markets (NCAAB, Serie A)	February 18, 2026	0.0175	1	0.44%	25%

18

Fees are collected asymmetrically depending entirely on the directional flow of the trade. For all buy orders crossing the spread, the fee is extracted directly in the form of outcome shares, slightly reducing the net position size acquired. Conversely, for all sell orders, the fee is extracted directly in USDC collateral from the total sale proceeds.[18] To prevent microscopic fractional accounting issues on the blockchain, the protocol enforces a strict minimum fee floor of one ten-thousandth of a USDC unit. Any mathematically calculated fee falling below this precise threshold is automatically rounded down to absolute zero.[21]

5.2. Mathematical Impact on Arbitrage Margins

The introduction of dynamic taker fees mandates that Python arbitrage algorithms must pre-calculate the exact impending fee drag in real-time before authorizing the execution of a Dutch Book transaction. Relying on a simplistic assessment where the sum of the Yes and No asks is less than one dollar will inevitably lead to net-negative executions when fees are applied.

Consider a practical scenario where an arbitrage bot detects an inefficiency in a daily crypto market. Both the Yes and No tokens are priced equally at exactly forty-eight and a half cents. Under the legacy zero-fee model, purchasing one hundred shares of each would require a capital outlay of ninety-seven dollars, resulting in a guaranteed risk-free profit of three dollars

upon event resolution.

However, under the 2026 fee paradigm, the calculation changes dramatically. Because the price is incredibly close to the fifty-cent midpoint, the effective fee rate approaches its maximum theoretical peak of 1.56%.

The algorithmic engine must compute the exact capital requirements:

1. The base cost for one hundred Yes shares is forty-eight dollars and fifty cents, plus an approximate dynamic fee of seventy-five cents, totaling forty-nine dollars and twenty-five cents.
2. The base cost for the corresponding one hundred No shares identically totals forty-nine dollars and twenty-5 cents.
3. The absolute total capital outlay required to execute the complete arbitrage cycle is ninety-eight dollars and fifty cents.
4. The mathematically guaranteed payout upon market resolution remains exactly one hundred dollars.
5. The true net arbitrage profit is reduced significantly to one dollar and fifty cents, rather than three dollars.

To remain profitable, trading bots must dynamically compute a comprehensive equation where the price of the Yes token, plus the price of the No token, plus the individually calculated fees for both tokens, plus the Polygon network gas execution costs, sum to a value strictly less than one dollar.

6. The Polymarket API Ecosystem and Data Pipelines

Polymarket's technical infrastructure is highly decentralized and intentionally decoupled into several specialized, autonomous Application Programming Interfaces. A professional-grade Python arbitrage bot must seamlessly integrate all of these disparate subsystems to function effectively, maintaining synchronized states across discovery, pricing, and execution layers.

6.1. The Gamma API for Market Discovery

The Gamma API serves as the foundational data discovery layer. Hosted at a dedicated endpoint, it is a publicly accessible, read-only RESTful service that continuously indexes raw on-chain UMA oracle data and enriches it with extensive market metadata.[22] It acts as the primary computational engine for discovering newly deployed tradable assets and fetching the critical ERC1155 Token IDs that are strictly required for any interaction with the Central Limit Order Book.[5]

To continuously fetch all live, actively tradable markets for cross-market combinatorial arbitrage scanning, the Python execution engine must systematically query the primary events endpoint. A standard REST GET request utilizes comprehensive query parameters to filter the massive dataset, requesting only active markets that have not yet closed, while paginating the

results to handle the payload size.[24]

The JSON response provided by the Gamma API yields an incredibly dense, nested structural array. The highest-level object is the Event, which contains an embedded array of associated Markets. Each individual Market object contains a deeply nested array of tokens, representing the exact sixty-four character alphanumeric strings required to identify the Yes and No tokens.[10] Crucially, the response payload also includes boolean flags, such as the enableOrderBook identifier. If this specific flag returns as false, it indicates that while the asset exists as a cryptographic token on the Polygon blockchain, it is currently unsupported by the off-chain matching engine and thus cannot be arbitraged via the CLOB API.[10]

6.2. The Central Limit Order Book Trading API

While the Gamma API handles discovery, the CLOB API is the exclusive conduit for active trading. It manages all live pricing states, order book depth arrays, and the entire lifecycle of order management.[22] While specific endpoints providing midpoints, spreads, and historical pricing trends remain public, any endpoint associated with order placement, order cancellation, or position querying requires rigorous, multi-layered cryptographic authentication.[22]

This authentication architecture operates strictly on two distinct cryptographic levels.[4] The first level involves an Ethereum Improvement Proposal 712 signature. The Python execution bot must utilize the private key of the funding wallet to mathematically sign a complex typed data structure. This specific message attests cryptographically that the algorithmic entity definitively controls the specified blockchain wallet.[5] This Level 1 signature is transmitted securely to the CLOB servers to generate or derive a suite of Level 2 API credentials.[4]

The second level of authentication utilizes standard HMAC-SHA256 encryption methodologies. The previously derived API credentials, comprising an API Key, an API Secret, and an API Passphrase, are utilized by the Python script to generate unique HMAC signatures for every single subsequent REST request.[4] The HTTP headers for any order placement must meticulously include the user's wallet address, the generated HMAC signature, a precision UNIX timestamp, the API key, and the passphrase.[4]

Crucially, even with a perfectly configured L2 HMAC authentication tunnel actively maintaining the session, the actual order payload—representing the exact price and size of the limit order—must still be individually hashed and signed using the EIP-712 standard via the wallet's private key.[4] This dual-layer architecture ensures that the centralized matching engine operator can never forge, alter, or spoof a user's trade without possessing the underlying private key.[4]

6.3. API Rate Limits and System Throttling

High-frequency algorithmic requests targeting the Polymarket infrastructure are subjected to highly stringent, tiered rate limiting protocols designed to preserve system stability. In early

2026, alongside the fee rollout, Polymarket drastically updated these thresholds.[19]

The public order book endpoints face significant throttling. Requests targeting the /books endpoint are strictly capped at three hundred requests per ten-second rolling window.[19] More critically for arbitrage execution, the primary POST /order endpoint utilizes a complex two-tier limitation system. Bots are permitted a maximum burst capacity of five hundred order placements every ten seconds, equating to fifty requests per second.[19] However, they are simultaneously constrained by a sustained limitation of three thousand order placements every ten minutes, which mathematically averages to only five requests per second over prolonged operational durations.[19]

The data analytics and discovery APIs possess their own unique limitations. The general Gamma API endpoints are restricted to fifteen thousand requests per ten seconds, while endpoints querying active user positions and historical trade data are heavily throttled to just one hundred and fifty requests per ten-second interval.[28]

7. Engineering the Python Execution Engine

Constructing an institutional-grade arbitrage bot requires bypassing raw HTTP requests and directly integrating the official py-clob-client SDK, a library maintained directly by Polymarket engineering.[2] This client completely abstracts the highly complex EIP-712 order signing mechanisms and the continuous generation of HMAC headers, allowing quantitative developers to focus computational resources entirely on strategy logic and execution latency.[4]

7.1. Client Initialization and Wallet Topography

The bot requires an environment running Python 3.9 or higher and the successful installation of the client package via standard pip commands.[2] To authenticate the session, the bot must initialize the ClobClient class by passing the target host URL, the Polygon Chain ID, the private key of the operational wallet, and the specific address acting as the funder.[2]

The signature_type parameter is absolutely critical during this initialization phase, and its exact integer value depends entirely on the specific cryptographic architecture of the wallet being utilized by the bot.[2]

Wallet Architecture	**Signature Type Integer**	**Operational Description**	**Funder Address Configuration**
Externally Owned	0	Standard cryptographic wallet.	The direct public address of the EOA

Account (EOA)		The bot pays its own execution gas in POL tokens.	wallet.
Polymarket Proxy Wallet	1	Accounts instantiated via Magic Link or Email login protocols.	The assigned proxy wallet address, not the signing key.
Gnosis Safe Contract Wallet	2	Advanced multi-signature or relayer-deployed smart contract wallets.	The proxy address of the Gnosis Safe.

2

Upon instantiating the client object with the correct signature methodology, the script executes the create_or_derive_api_creds() function, captures the returned payload, and permanently sets the active credentials utilizing set_api_creds(), fully authenticating the execution engine for the duration of the process lifecycle.[2]

7.2. Managing On-Chain State and Token Allowances

A pervasive point of catastrophic failure for novice algorithmic developers utilizing standard EOA wallets involves attempting to dispatch signed orders without first granting the Polymarket Exchange contracts permission to mathematically manipulate and transfer the user's tokens. Unlike highly centralized cryptocurrency exchanges where deposits are custodial, Polymarket's non-custodial settlement mechanism strictly requires standard ERC-20 approve transactions and ERC-1155 setApprovalForAll transactions to be successfully broadcast and confirmed directly on the Polygon blockchain before the off-chain matching engine will accept any signed order payload.[2]

The bot must possess sufficient balances of two specific token types: the bridged USDC collateral token and the specific Conditional Token Framework outcome tokens representing the actual prediction market shares.[31] These specific tokens must be explicitly approved for three completely separate smart contract targets to ensure comprehensive operational access to both standard binary markets and complex Negative Risk events.[2]

Spender Contract Target	Polygon Deployment Address	Functional Purpose for Arbitrage
Core CTF Exchange	0x4bFb41d5B3570DeFd03C39a9A4D8dE6Bd8B8982E	Settlement and clearance of standard independent markets.
Negative Risk CTF Exchange	0xC5d563A36AE78145C45a50134d48A1215220f80a	Order matching and execution for linked multi-outcome markets.
Negative Risk Adapter	0xd91E80cF2E7be2e162c6513ceD06f1dD0dA35296	Executing the atomic conversion of No tokens into complementary Yes tokens.

16

These foundational approvals are executed via on-chain state changes utilizing a standard Web3 Python library. The script must initialize the contract interfaces using the specific Application Binary Interfaces of the USDC and CTF contracts, build the raw transaction payloads specifying the target addresses and maximum integer allowance values, sign them using the bot's private key, and utilize the wait_for_transaction_receipt methodology to ensure absolute blockchain finality before transitioning into the active trading loop.[2] Accounts utilizing the Polymarket Proxy infrastructure or Magic Link logins bypass this requirement, as the internal proxy architecture handles allowance management automatically.[2]

7.3. Order Routing and Tick Size Enforcement

When a pricing inefficiency is detected, the speed and accuracy of the order execution engine dictate profitability. All orders submitted to Polymarket, including aggressive market orders designed to cross the spread immediately, are mathematically expressed as limit orders within the API payload.[33] To execute a market order, the Python script must submit a limit order formulated with a highly marketable price, explicitly commanding the matching engine to execute the trade instantly against the best available resting liquidity.[33]

The specific behavior of these orders is parameterized strictly within the py-clob-client utilizing the OrderType enumeration.[33] For capturing a fleeting Dutch Book arbitrage opportunity that

currently rests transparently on the order book, the execution engine must never utilize standard Good-Til-Cancelled order types. Instead, the bot must utilize the Fill-Or-Kill or Fill-And-Kill parameters.[2]

The Fill-Or-Kill parameter mandates that the submitted order must be filled entirely in its specified size, and it must occur immediately. If the targeted liquidity has been partially consumed by a competing high-frequency bot in the preceding milliseconds, the entire Polymarket matching engine aborts the transaction, returning an unfilled status.[33] This specific order type is absolutely mandatory for pure arbitrage to strictly prevent the catastrophic scenario known as a "legged" trade, wherein the bot successfully acquires the Yes token but simultaneously fails to acquire the complementary No token, leaving the algorithm fully exposed to directional market risk rather than remaining market-neutral.[33]

The order payload is constructed utilizing the OrderArgs dataclass, requiring the target price, the desired size in shares, the trade direction, and the specific Token ID.[29] This dataclass is then passed sequentially to the create_and_post_order function.[29] Crucially, due to strict validation protocols within the matching engine, if the target market is structurally part of a Negative Risk event, a specific negRisk boolean must be explicitly declared and passed within the PartialCreateOrderOptions object. Failure to include this precise parameter results in the matching engine returning a 400 status code indicating an invalid signature payload, causing the execution cycle to fail catastrophically.[35]

Furthermore, the execution logic must strictly adhere to the specific minimum price increments enforced by the exchange, known as tick sizes. If a bot attempts to submit a signed order payload priced at forty-eight point one cents in a market constrained by a tick size of exactly one cent, the Central Limit Order Book will reject the cryptographic signature outright.[33] The Python engine must dynamically read the tick_size parameter directly from the order book data payload and rigorously round all mathematically calculated execution prices to the acceptable floating-point precision format prior to initiating the EIP-712 serialization process.[25]

8. Low-Latency Data Streaming and Market Making Arbitrage

Due to the highly constrained API rate limits discussed previously, relying on REST API polling methodologies to continuously download the order book state is technologically unviable for a high-frequency Python execution bot.[19] To remain competitive, the architecture must immediately transition to the WebSocket API infrastructure for all live data ingestion.[2]

8.1. WebSocket Subscription Topography

The execution engine must establish a persistent, asynchronous connection to the primary

market channel located at wss://ws-subscriptions-clob.polymarket.com/ws/market.[36] Upon completing the TCP handshake, the script formats and transmits a heavily structured JSON subscription payload containing a specific array of Token IDs, denoted as assets_ids, representing the exact markets the algorithm intends to monitor.[36]

The subscription payload allows quantitative developers to request highly specific data stream variants. While subscribing to the book stream provides full, continuous orderbook snapshots, this is highly bandwidth-intensive. Professional execution engines primarily subscribe to the price_change stream, which delivers highly optimized, tick-by-tick level updates, and the last_trade_price stream, which confirms actual market executions.[36]

Most critically for instantaneous Yes/No sum arbitrage, the Python script must explicitly activate the custom_feature_enabled: true flag within the subscription payload. This specific boolean activates access to the best_bid_ask event stream.[36] This proprietary stream pushes updates to the client in milliseconds whenever the absolute top of the order book changes, allowing the bot's internal logic loop to instantaneously recalculate the Dutch Book threshold without parsing the entire depth array.[36]

To prevent the centralized servers from terminating the persistent WebSocket connection during periods of low market volatility, the Python bot must periodically dispatch a specialized HTTP POST request to the /heartbeats endpoint.[27] This specific request must be fully authenticated utilizing the exact same Level 2 HMAC headers required for order placement, signaling to the server that the automated client remains actively engaged and computationally responsive.[5]

8.2. Market Making as Hedged Arbitrage

While pure taker arbitrage involves aggressively buying existing asks to capture inefficiencies, this methodology suffers from high slippage, continuous gas expenditures, and the crippling impact of the 2026 dynamic taker fees.[18] Advanced algorithmic traders utilizing Python deploy a far more complex strategy: Maker Arbitrage, effectively acting as high-frequency market makers.

Instead of crossing the spread and incurring fees, the algorithmic engine simultaneously places highly calculated passive bids for both the Yes and No tokens directly onto the order book, aiming to capture the spread differential rather than paying it.[13] To ensure the bot acts solely as a liquidity provider, it utilizes specialized Post-Only Orders.[33] By appending a postOnly = true command to a standard Good-Til-Cancelled limit order via the py-clob-client, the bot guarantees cryptographically that if its specified limit price would inadvertently cause an immediate match—thereby crossing the spread and triggering a taker fee—the order is immediately rejected by the exchange rather than executed.[33]

This methodology transforms the execution engine into a spread capture mechanism.

However, market making requires incredibly strict inventory management parameters. If the algorithm's Yes bids are consistently filled by market participants while its corresponding No bids remain ignored on the book, the bot rapidly accumulates directional market risk, abandoning its neutral arbitrage mandate.[13] The Python script must actively monitor its accumulated inventory via the authenticated User WebSocket channel. Upon detecting an imbalance, it must systematically manage its skew by immediately lowering its Yes bid price and aggressively repricing its No ask to entice the market to unwind the excess inventory.[13]

As a massive structural incentive for deploying this specific strategy, Polymarket redistributes a significant percentage of the collected taker fees back to active market makers daily. These Maker Rebates, distributed securely in USDC, range from twenty to twenty-five percent depending on the specific market classification, materially augmenting the base yield generated by the algorithmic spread capture.[19]

9. Quantitative Risk Management and Oracle Settlement Architecture

Arbitrage on decentralized prediction platforms is strictly defined as mathematically risk-free solely under the absolute assumption that the underlying smart contract infrastructure functions flawlessly and that the decentralized oracle correctly resolves the event according to the exact, explicitly written parameters of the market.[5]

9.1. Capital Lockup and Opportunity Costs

A fundamental operational risk managed by the algorithmic engine is the duration of capital lockup. Arbitrageurs who build a successful Dutch Book position, guaranteeing a one dollar payout, must physically hold those tokens in their cryptographic wallet until the real-world event actually concludes and the oracle finalizes the resolution data.[39] If the market targeted by the bot is a long-duration event, such as a geopolitical election scheduled two years in the future, the opportunity cost of that locked USDC capital may vastly exceed the three to five percent arbitrage yield generated.[39]

Sophisticated Python bots circumvent this capital inefficiency by engaging in Secondary Market Unwinding. If the algorithmic engine acquires a synthetic one-dollar position for exactly ninety-five cents, it does not strictly need to wait for the final oracle resolution if the secondary market corrects its pricing inefficiency. If the combined order book bids for the Yes and No tokens organically rise to ninety-nine cents, the bot is programmed to instantly sell its accumulated inventory back into the Central Limit Order Book, realizing an immediate four-cent profit and freeing its USDC capital reserves for subsequent high-velocity execution cycles.[13]

9.2. Resolution Disputes and Gas Optimization

Polymarket relies fundamentally on the UMA Optimistic Oracle for market resolution, a decentralized system that relies on economic incentives and a bonded dispute period to verify real-world data.[16] In highly contentious markets, oracle disputes can trigger massive delays in capital redemption. In extreme scenarios where the oracle mechanism fails or multiple conflicting conditions resolve simultaneously, Polymarket relies on a specialized permissioned multisignature contract known as the NegRiskOperator.[40] This specific smart contract grants administrative accounts the ability to flag individual questions, preventing normal resolution, and allows for manual emergency resolution after a strict time delay.[40] The execution engine must factor the tail-risk of disputed resolutions into its position sizing matrices.

Furthermore, while Polymarket's off-chain matching is entirely gas-free, the critical on-chain settlement of trades, the complex splitting and merging of Conditional Token Framework tokens, and the execution of Negative Risk conversions via the NegRiskAdapter fundamentally require continuous expenditures of Polygon gas.[4] To completely circumvent fluctuating gas fees and the massive operational complexities of managing MATIC or POL collateral balances across multiple autonomous bot wallets, institutional arbitrageurs deploy sophisticated Gnosis Safe proxy wallets utilizing Polymarket's specific Relayer SDK.[2]

When integrating the Python Relayer client, the execution bot signs the transaction intent cryptographically, but Polymarket's backend relayer server actually assumes the responsibility of submitting the raw transaction to the Polygon blockchain.[32] By doing so, the protocol completely subsidizes the gas costs for the algorithmic user, removing a highly variable cost from the complex mathematical equation governing arbitrage profitability and materially widening the statistical window for profitable executions.[32]

By successfully integrating these disparate components—from low-latency WebSocket ingestion and dynamic fee calculations to gasless relayer infrastructure and Negative Risk conversions—a quantitative developer can deploy a highly resilient, market-neutral Python execution engine fully capable of extracting systemic yield from the world's largest decentralized prediction marketplace.

Works cited

1. Polymarket Documentation: Overview, accessed March 8, 2026, https://docs.polymarket.com/
2. Python client for the Polymarket CLOB - GitHub, accessed March 8, 2026, https://github.com/Polymarket/py-clob-client
3. Polymarket funding, news & analysis | Sacra, accessed March 8, 2026, https://sacra.com/c/polymarket/
4. Overview - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/trading/overview
5. The Polymarket API: Architecture, Endpoints, and Use Cases - Medium, accessed March 8, 2026, https://medium.com/@gwrx2005/the-polymarket-api-architecture-endpoints-an

d-use-cases-f1d88fa6c1bf
6. How to Setup a Polymarket Bot: Step-by-Step Guide for Beginners | QuantVPS, accessed March 8, 2026, https://www.quantvps.com/blog/setup-polymarket-trading-bot
7. How Elite Coders Built Bots Earning $200K Monthly On Polymarket Without Ever Predicting Outcomes | Yellow Media on Binance Square, accessed March 8, 2026, https://www.binance.com/en/square/post/34013233971537
8. Polymarket Prediction Market Decryption: How to Price Probabilities Using Price Mechanisms | DA Labs on Binance Square, accessed March 8, 2026, https://www.binance.com/en/square/post/34041203415945
9. [2508.03474] Unravelling the Probabilistic Forest: Arbitrage in Prediction Markets - arXiv, accessed March 8, 2026, https://arxiv.org/abs/2508.03474
10. Markets & Events - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/concepts/markets-events
11. Conditional Token Framework - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/trading/ctf/overview
12. How to Programmatically Identify Arbitrage Opportunities on Polymarket (And Why I Built a Portfolio Betting Agent Instead) | by PolyMaster | Medium, accessed March 8, 2026, https://medium.com/@wanguolin/how-to-programmatically-identify-arbitrage-opportunities-on-polymarket-and-why-i-built-a-portfolio-23d803d6a74b
13. Automated Trading on Polymarket: Bots, Arbitrage & Execution Strategies - QuantVPS, accessed March 8, 2026, https://www.quantvps.com/blog/automated-trading-polymarket
14. A Polymarket and Kalshi Arbitrage bot written in Python, watches 10000+ markets looking for inefficient markets on and between platforms - GitHub, accessed March 8, 2026, https://github.com/ImMike/polymarket-arbitrage
15. Negative Risk Markets - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/advanced/neg-risk
16. Contract Addresses - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/resources/contract-addresses
17. Polymarket/neg-risk-ctf-adapter - GitHub, accessed March 8, 2026, https://github.com/Polymarket/neg-risk-ctf-adapter
18. Fees - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/trading/fees
19. Polymarket Changelog, accessed March 8, 2026, https://docs.polymarket.com/changelog
20. Polymarket quietly introduces taker fees on 15-minute crypto markets - TradingView, accessed March 8, 2026, https://www.tradingview.com/news/cointelegraph:e59c32089094b:0-polymarket-quietly-introduces-taker-fees-on-15-minute-crypto-markets/
21. Maker Rebates Program - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/market-makers/maker-rebates
22. Introduction - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/api-reference/introduction

23. polymarketgamma package - github.com/ivanzzeth/polymarket-go-gamma-client - Go Packages, accessed March 8, 2026, https://pkg.go.dev/github.com/ivanzzeth/polymarket-go-gamma-client
24. Fetching Markets - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/market-data/fetching-markets
25. Overview - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/market-data/overview
26. Authenticated (CLOB) - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/api-reference/authentication
27. Send heartbeat - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/api-reference/trade/send-heartbeat
28. Rate Limits - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/api-reference/rate-limits
29. py-clob-client - PyPI, accessed March 8, 2026, https://pypi.org/project/py-clob-client/0.1.1/
30. Quickstart - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/trading/quickstart
31. py-clob-client 0.24.0 - PyPI, accessed March 8, 2026, https://pypi.org/project/py-clob-client/0.24.0/
32. Getting Started - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/market-makers/getting-started
33. Overview - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/trading/orders/overview
34. Create Order - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/trading/orders/create
35. Invalid signature when creating orders in negative risk market · Issue #79 · Polymarket/py-clob-client - GitHub, accessed March 8, 2026, https://github.com/Polymarket/py-clob-client/issues/79
36. Overview - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/market-data/websocket/overview
37. Automated market making bot for Polymarket prediction markets - GitHub, accessed March 8, 2026, https://github.com/terrytrl100/polymarket-automated-mm
38. Code Assessment of the NegRiskAdapter Smart Contracts - ChainSecurity, accessed March 8, 2026, https://reports.chainsecurity.com/Polymarket/ChainSecurity_Polymarket_NegRiskAdapter_Audit.pdf
39. Polymarket Arbitrage and Airdrop Farming: How Traders Avoid Directional Bets - Reddit, accessed March 8, 2026, https://www.reddit.com/r/CryptoCurrency/comments/1r2gy87/polymarket_arbitrage_and_airdrop_farming_how/
40. neg-risk-ctf-adapter/src/NegRiskOperator.sol at main - GitHub, accessed March 8, 2026, https://github.com/Polymarket/neg-risk-ctf-adapter/blob/main/src/NegRiskOpera

tor.sol
41. neg-risk-ctf-adapter/docs/NegRiskOperator.md at main - GitHub, accessed March 8, 2026, https://github.com/Polymarket/neg-risk-ctf-adapter/blob/main/docs/NegRiskOperator.md
42. Clients & SDKs - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/api-reference/clients-sdks
43. Tiers - Polymarket Documentation, accessed March 8, 2026, https://docs.polymarket.com/builders/tiers